

Deep Learning

12. Recurrent Neural Networks

Dr. Konda Reddy Mopuri
Dept. of AI, IIT Hyderabad
Jan-May 2026

So far...

① Perceptron, MLP, Gradient Descent (Backpropagation)

So far...

- ① Perceptron, MLP, Gradient Descent (Backpropagation)
- ② CNNs

So far...

- ① Perceptron, MLP, Gradient Descent (Backpropagation)
- ② CNNs
- ③ 'Feedforward Neural networks'

Feedforward NNs: some observations

- ① Size of the i/p is fixed(?!)

Feedforward NNs: some observations

- ① Size of the i/p is fixed(?!)
- ② Successive i/p are i.i.d.

Feedforward NNs: some observations

- ① Size of the i/p is fixed(!)
- ② Successive i/p are i.i.d.
- ③ Processing of successive i/p is independent of each other

Consider 'auto-completion' task

Q deep|

deep — Search with Google

🕒 **kuldeep birdar**

Q **deepika padukone**

Q **deepthi sunaina**

Q **deepak bagga**

Q **deepika pilli**

Q **deepthi sharma**

① Successive i/p are not independent

Consider 'auto-completion' task

🔍 deep|

🔍 deep — Search with Google

- 🕒 **kuldeep birdar**
- 🔍 **deepika padukone**
- 🔍 **deepthi sunaina**
- 🔍 **deepak bagga**
- 🔍 **deepika pilli**
- 🔍 **deepthi sharma**

- ① Successive i/p are not independent
- ② Length of the i/p is not fixed (→ predictions also)

Consider 'auto-completion' task

🔍 deep|

🔍 deep — Search with Google

- 🕒 **kuldeep birdar**
- 🔍 deep**pika padukone**
- 🔍 deep**thi sunaina**
- 🔍 deep**pak bagga**
- 🔍 deep**pika pilli**
- 🔍 deep**ti sharma**

- ① Successive i/p are not independent
- ② Length of the i/p is not fixed (→ predictions also)
- ③ Same underlying task at different 'time instances'

Consider 'auto-completion' task

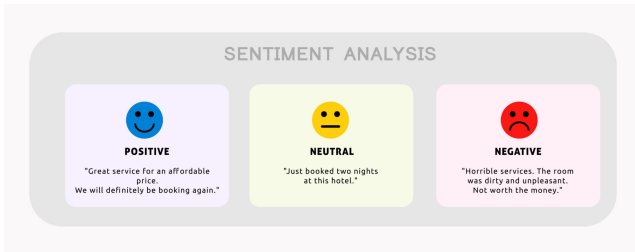
deep|

deep — Search with Google

- 🕒 **kuldeep birdar**
- 🔍 deep**pika padukone**
- 🔍 deep**thi sunaina**
- 🔍 deep**pak bagga**
- 🔍 deep**pika pilli**
- 🔍 deep**ti sharma**

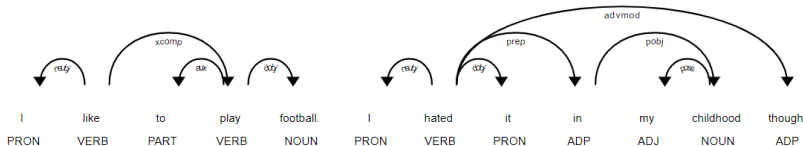
- ① Successive i/p are not independent
- ② Length of the i/p is not fixed (→ predictions also)
- ③ Same underlying task at different 'time instances'
- ④ **Sequence Learning Problems**

Sequence Learning Tasks: Example



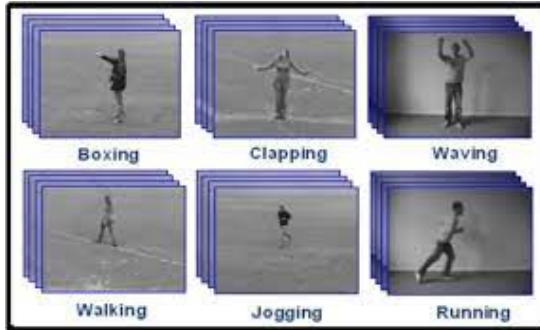
Sentiment Analysis (Source)

Sequence Learning Tasks: Example



POS-Tagging (Source:Kaggle)

Sequence Learning Tasks: Example



Action Recognition (Source)

Sequence Learning Tasks: Example

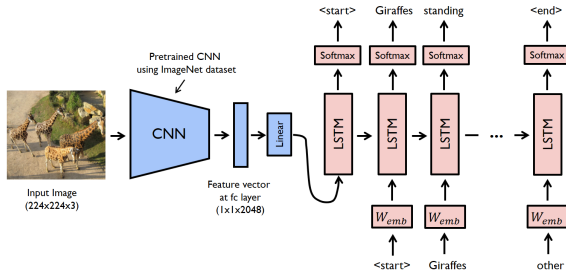
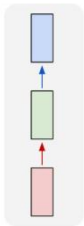


Image Captioning(Source)

Sequence Learning Tasks: Variations

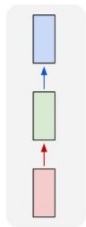
one to one



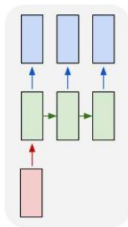
Source

Sequence Learning Tasks: Variations

one to one

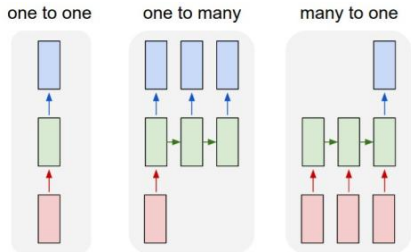


one to many



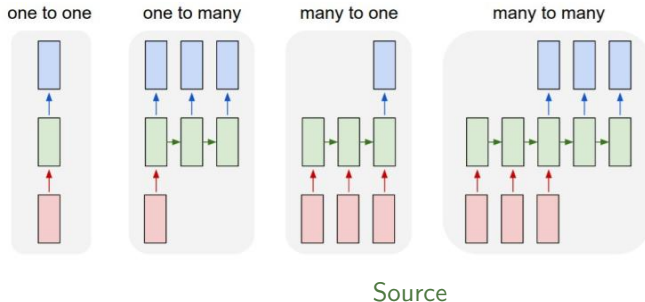
Source

Sequence Learning Tasks: Variations

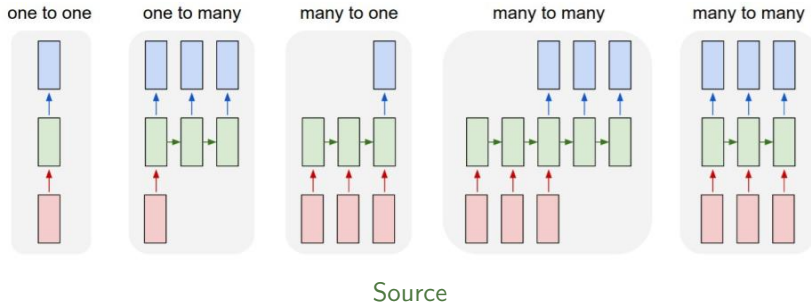


Source

Sequence Learning Tasks: Variations



Sequence Learning Tasks: Variations



Recurrent Neural Networks (RNN)

- ① NNs designed to solve sequence learning tasks

Recurrent Neural Networks (RNN)

- ① NNs designed to solve sequence learning tasks
- ② Characteristics
 - ① Model the dependence among the i/p

Recurrent Neural Networks (RNN)

- ① NNs designed to solve sequence learning tasks
- ② Characteristics
 - ① Model the dependence among the i/p
 - ② Handle variable length of i/p

Recurrent Neural Networks (RNN)

- ① NNs designed to solve sequence learning tasks
- ② Characteristics
 - ① Model the dependence among the i/p
 - ② Handle variable length of i/p
 - ③ Same function applied at all time instances

Can we not use the tools we already know?

Can we not use the tools we already know?

- How about convolution?

Time-delay Neural Network

- Sliding predictor

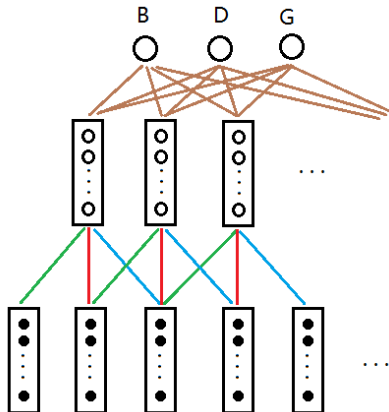
Time-delay Neural Network

- Sliding predictor
- Looks at the last few 'time steps'

Time-delay Neural Network

- Sliding predictor
- Looks at the last few 'time steps'
- This is a CNN applied to sequence data

Time-delay Neural Network



Finite-response Model

- I/p at a time step affects the o/p only till the N steps into the future

Finite-response Model

- I/p at a time step affects the o/p only till the N steps into the future
- N - width of the system

Finite-response Model

- I/p at a time step affects the o/p only till the N steps into the future
- N - width of the system
- $Y_t = f(X_t, X_{t-1}, \dots, X_{t-N})$

Finite-response Model

- What is an ideal width/history to consider?

Finite-response Model

- What is an ideal width/history to consider?
- Trends can be short-term or long-term

Infinite memory

- Infinite response systems

Infinite memory

- Infinite response systems
- I/p at a timestep can continue to affect the o/p forever

Infinite memory

- Infinite response systems
- I/p at a timestep can continue to affect the o/p forever
- Possibly with weaker and weaker influence

- Infinite response systems
- I/p at a timestep can continue to affect the o/p forever
- Possibly with weaker and weaker influence
- $Y_t = f(X_t, X_{t-1}, \dots, X_{t-\infty})$

Example Infinite response systems

Example Infinite response systems

- $Y_t = f(X_t, Y_{t-1})$

Example Infinite response systems

- $Y_t = f(X_t, Y_{t-1})$
- Define initial state Y_0

Example Infinite response systems

- $Y_t = f(X_t, Y_{t-1})$
- Define initial state Y_0
- An instance of a NARX (Nonlinear Autoregressive Network with Exogenous inputs) system

Example Infinite response systems

- $Y_t = f(X_t, Y_{t-1})$
- Define initial state Y_0
- An instance of a NARX (Nonlinear Autoregressive Network with Exogenous inputs) system
- More general one, $Y_t = f(X_{0:t}, Y_{0:t-1})$

- Popular for time-series prediction

NARX systems

- Popular for time-series prediction
- E.g., Weather, Stock markets, Language, etc.

- Popular for time-series prediction
- E.g., Weather, Stock markets, Language, etc.
- Memory of the past (or, history) is in the output itself, not in the network

Jordan Network (1986)

- Memory is a running average of the past outputs ($m_t = Y_t + \rho \cdot m_{t-1}$)

Jordan Network (1986)

- Memory is a running average of the past outputs ($m_t = Y_t + \rho \cdot m_{t-1}$)
- Memory has a fixed structure, not learning-driven

Memory is not explicit

- Output may be a harsh summary (not enough as a memory)

Memory is not explicit

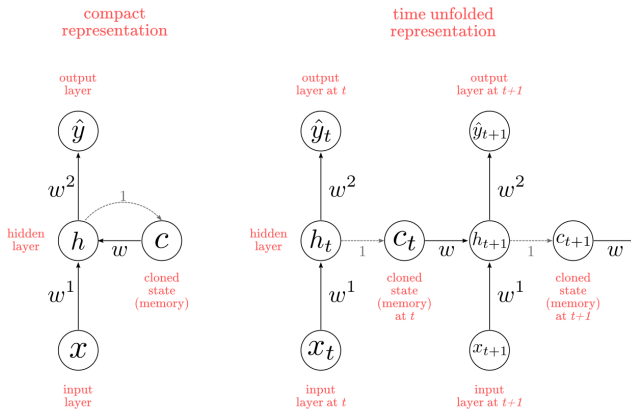
- Output may be a harsh summary (not enough as a memory)
- Information loss

Memory is not explicit

- Output may be a harsh summary (not enough as a memory)
- Information loss
- May want to remember what happened in the network

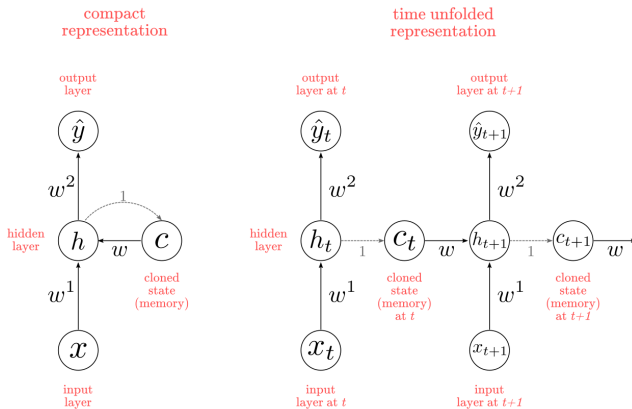
Elman Network (1990)

- Introduced a separate memory state via context units



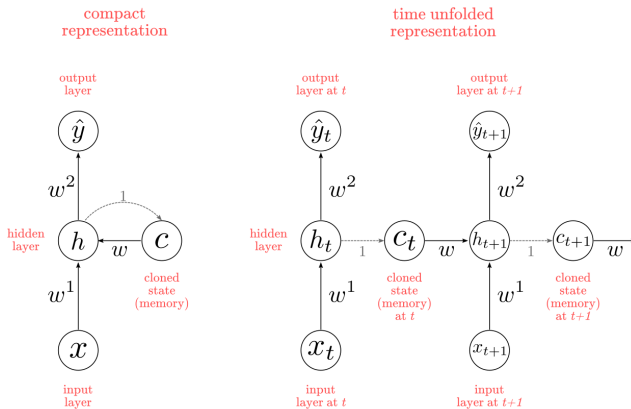
Elman Network (1990)

- Introduced a separate memory state via context units
- They clone the hidden response from previous timestep



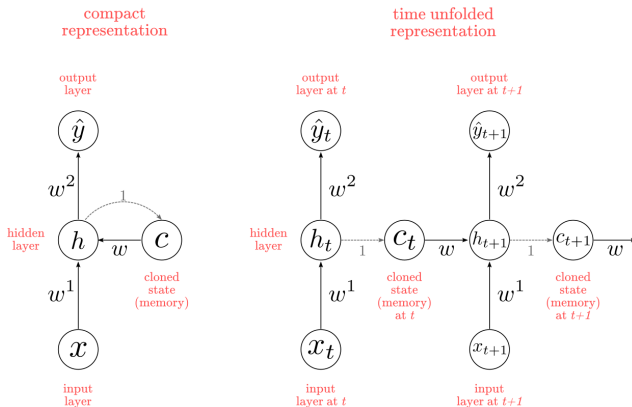
Elman Network (1990)

- Introduced a separate memory state via context units
- They clone the hidden response from previous timestep
- For training, approximated as a set of N independent 1-step history nets



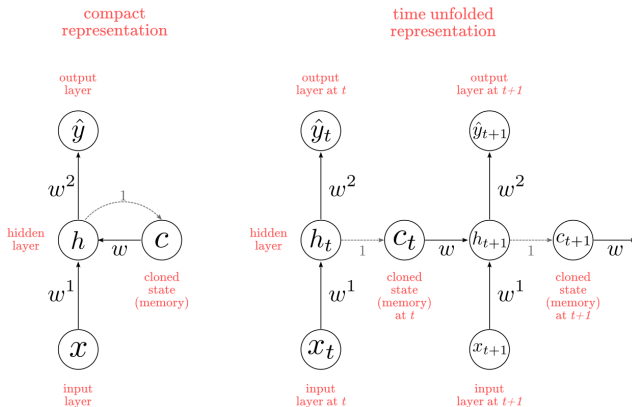
Elman Network (1990)

- Gradient doesn't backprop beyond the link with weight '1'



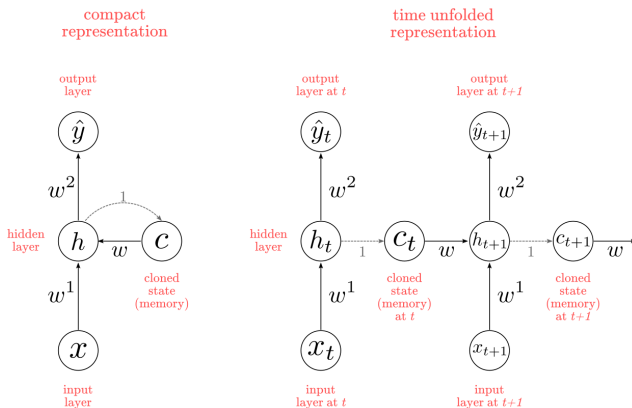
Elman Network (1990)

- Gradient doesn't backprop beyond the link with weight '1'
- Long-term dependencies are not learned :- (



Elman Network (1990)

- Gradient doesn't backprop beyond the link with weight '1'
- Long-term dependencies are not learned :-
- Known as SRN (Simple Recurrent Networks)



Toward an infinite response system

① State-space model

Toward an infinite response system

- ① State-space model
- ② $h_t = f(x_t, h_{t-1})$, state of the network (h_0 needs to be defined)

Toward an infinite response system

- ① State-space model
- ② $h_t = f(x_t, h_{t-1})$, state of the network (h_0 needs to be defined)
- ③ It summarizes the entire past (model directly embeds memory in the state)

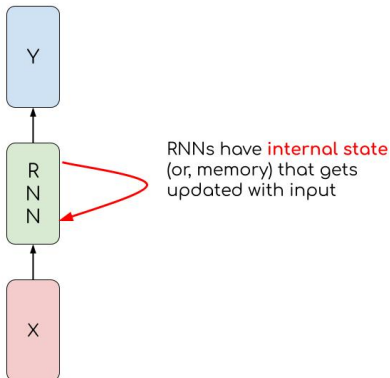
Toward an infinite response system

- ① State-space model
- ② $h_t = f(x_t, h_{t-1})$, state of the network (h_0 needs to be defined)
- ③ It summarizes the entire past (model directly embeds memory in the state)
- ④ $y_t = g(h_t)$, output

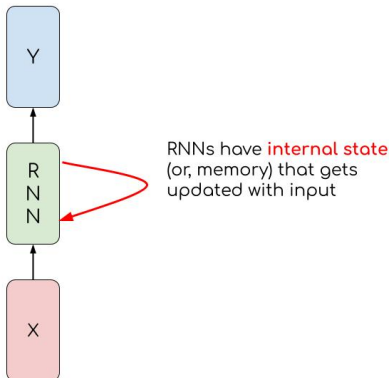
Toward an infinite response system

- ① State-space model
- ② $h_t = f(x_t, h_{t-1})$, state of the network (h_0 needs to be defined)
- ③ It summarizes the entire past (model directly embeds memory in the state)
- ④ $y_t = g(h_t)$, output
- ⑤ Known as 'Fully Recurrent Neural Networks', simply Recurrent neural Networks (RNN)

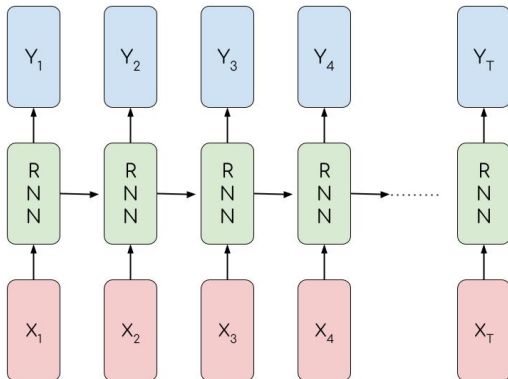
Toward an infinite response system



RNNs: internal state



RNNs: unfolding



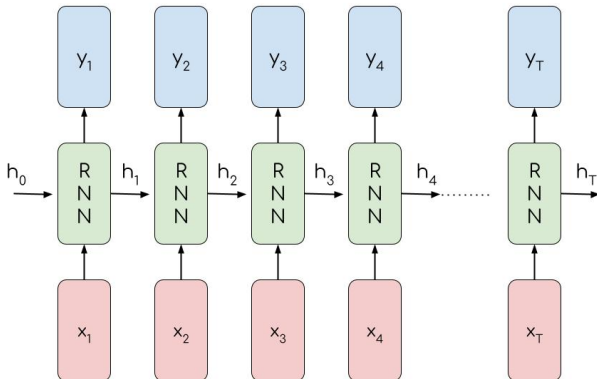
- ① Apply the same transformation at every time step $\rightarrow$ 'Recurrent' NNs

- ① Apply the same transformation at every time step $\rightarrow$ 'Recurrent' NNs
- ② i/p sequence $x_t \in \mathbb{R}^D$

- ① Apply the same transformation at every time step $\rightarrow$ 'Recurrent' NNs
- ② i/p sequence $x_t \in \mathbb{R}^D$
- ③ Initial recurrent state $h_0 \in \mathbb{R}^Q$

- ① Apply the same transformation at every time step $\rightarrow$ 'Recurrent' NNs
- ② i/p sequence $x_t \in \mathbb{R}^D$
- ③ Initial recurrent state $h_0 \in \mathbb{R}^Q$
- ④ RNN model computes sequence of recurrent states iteratively
$$h_t = \phi(x_t, h_{t-1}; w)$$

RNNs



RNN (1990)

- ① Start with $h_0 = 0$

RNN (1990)

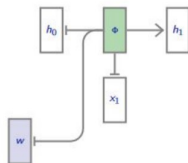
- ① Start with $h_0 = 0$
- ② $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$

RNN (1990)

- ① Start with $h_0 = 0$
- ② $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$
- ③ $y_t = \text{softmax}(W_{hy} \cdot h_t + b_y)$

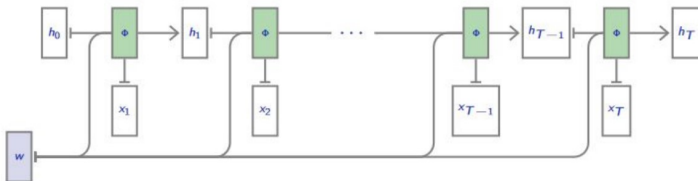
RNNs as computational graph

- 1 Use the same set of parameters at each time step



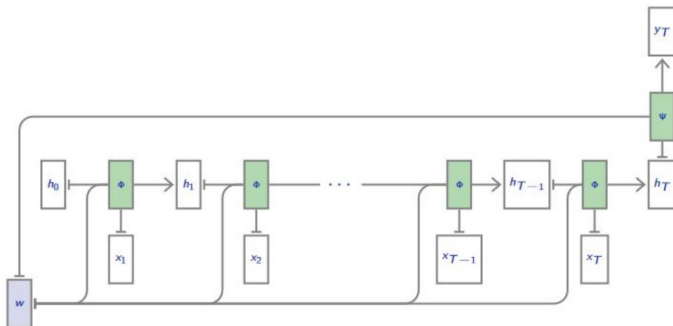
RNNs as computational graph

- ① Use the same set of parameters at each time step



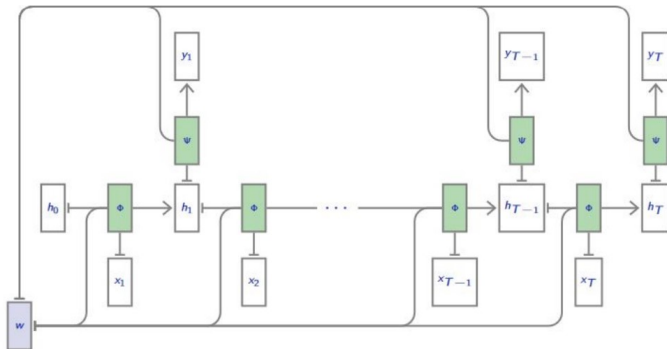
RNNs as computational graph

- 1 Use the same set of parameters at each time step



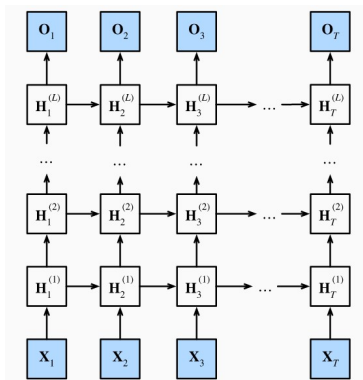
RNNs as computational graph

- 1 Use the same set of parameters at each time step
- 2 Flexible to realize different variants (with some tricks!)



Multi-layered RNNs

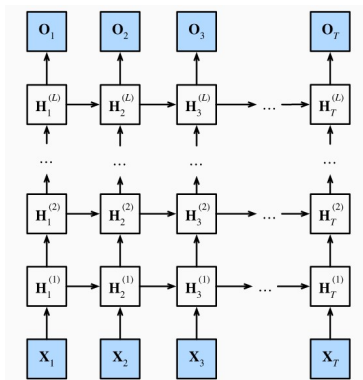
- 1 Stack multiple RNNs between i/p and o/p layers



Source

Multi-layered RNNs

- 1 Stack multiple RNNs between i/p and o/p layers
- 2 $H_t^{(l)} = W_{xh}^{(l)} \cdot H_t^{(l-1)} + W_{hh}^{(l)} \cdot H_{t-1}^{(l)} + b_h^{(l)}$



Source

- ① Continue with sequence models (RNNS, LSTM, etc.)